DIKAI DC810 QR Build And Send Process

Manufacturer technical summary and support questions

Scope: this document explains only how our device builds QR data and sends it to the DIKAI DC810 printer over TCP. It also lists the exact guidance we need from the manufacturer to improve QR print quality.

1. Current Implementation Summary

Item	Current behavior
Printer model	DIKAI DC810
Communication	TCP to printer IP/port, using LT-style framed packets
Dynamic text	Sent to Net Text field 1 with command 'T'
Dynamic QR	Generated by our device as a software QR bitmap, then sent to Net Logo slot 1 with command 'L'
Current QR issue	QR sent as Net Logo prints with poor quality and cannot be scanned reliably
Data length	The QR string is often longer than 30 characters
Main limitation	Net Logo does not give the operator easy QR customization options on the printer

Important: currently we are not sending data to the printer's 2D Code object. Our application builds the QR itself and uploads it as a bitmap/logo. We want to know if the 2D Code object can receive dynamic data directly from our device, because the printer's 2D Code editor has better QR customization options.

2. Process Flow

- Operator enters carton fields in the application: item code, lot number, shift, date/time, and serial number.
- The application builds one QR payload string using the format ITEM_CODE|LOT_NUMBER|SHIFT|DATE TIME|SN.
- When relevant form fields change, the application calls PrinterLink.push_label(...).
- The application sends the QR payload string to Net Text field 1 using the 'T' command.
- The application waits 100 ms.
- The application uses the qrcode library to render the QR payload into a square 0/1 dot matrix.
- The app caps the QR bitmap at the effective print area: min(34, QR_MAX_DOTS, MSG_PRINTED_DOTS). Current effective cap is 30 dots.
- The app flattens the matrix row-by-row into ASCII '1' and '0' bits.
- The app sends the QR bitmap to Net Logo slot 1 using the 'L' command.
- The printer then prints the active message containing Net Text and Net Logo when Jet Start and Print Enable are active.

3. QR Payload Construction

This is the exact data string encoded into the QR.

```
def build_qr_payload(self) -> str:
    dt = (self.batch_date or datetime.now()).strftime("%d %b %y")
    return f"{self.item_code}|{self.lot_number}|{self.shift}|{dt} {self.batch_time}|{self.sn}"
```

```
Format:
ITEM_CODE|LOT_NUMBER|SHIFT|DATE TIME|SN

Example:
AAS60601|LOT-1|M|24 Jun 26 08:00 AM|42
```

4. Trigger That Sends Data To Printer

When the QR-related form fields change, this block pushes the new payload.

```
def _on_form_changed(self, carton: CartonLabel):
    self.preview.update_preview(carton)

    sig = (
        carton.brand, carton.item_code, carton.item_desc,
        carton.lot_number, carton.sn, carton.grade, carton.shift,
        carton.size_code, carton.pcs_type, carton.pcs_per_ctn,
        carton.lpn_id, carton.carton_code,
    )
    if sig == getattr(self, "_last_push_sig", None):
        return
    self._last_push_sig = sig

    self.printer.push_label(
        text_payload=carton.brand,
        qr_payload=carton.build_qr_payload()
    )
```

5. QR Bitmap Generation Before Sending

The printer is receiving a bitmap/logo, not a native QR command. This code creates the QR dots.

```
def build_printer_bitmap(payload: str, max_size: int = 34, border: int = 2,
                         error_correction: str = "L"):
    ecc = _resolve_ecc(error_correction)
    last_n = None

    for b in range(max(border, 0), -1, -1):
        qr = qrcode.QRCode(
            version=None,
            error_correction=ecc,
            box_size=1,
            border=b,
        )
        qr.add_data(payload or " ")
        qr.make(fit=True)
        matrix = qr.get_matrix()
        n = len(matrix)
        last_n = n

        if n <= max_size:
            bits = "".join("1" if c else "0" for row in matrix for c in row)
            return n, n, bits

    raise ValueError(
        f"QR for {len(payload)} chars needs {last_n}x{last_n} dots, exceeds {max_size}."
    )
```

Current effective QR size cap used before bitmap generation:

```
msg_print_dots = int(getattr(config_app, "MSG_PRINTED_DOTS", MAX_DOTS))
qr_max = max(1, min(
    MAX_DOTS,                                    # 34 dot printer ceiling
    int(getattr(config_app, "QR_MAX_DOTS", MAX_DOTS)),
    msg_print_dots if msg_print_dots > 0 else MAX_DOTS,
))
```

6. Packet Layout For Net Text And Net Logo

All outgoing commands are framed as [0x10][PKGID][COMMAND][DATA...][0x01].

```
SEND_STX = 0x10
SEND_ETX = 0x01

def _frame(pkgid: int, type_byte: int, data: bytes = b"") -> bytes:
    return bytes([SEND_STX, pkgid, type_byte]) + data + bytes([SEND_ETX])
```

Net Text command. This sends the QR payload string to Network Text field 1.

```
def build_update_network_text(pkgid: int, text_id: int, text_value: str) -> bytes:
    text_value = (text_value or " ")[:100]
    text_id = max(1, min(10, int(text_id)))
    data = bytes([text_id]) + text_value.encode("ascii", errors="replace")
    return _frame(pkgid, ord("T"), data)
```

Net Logo command. This sends the QR bitmap to Logo slot 1.

```
def build_net_logo_packet(pkgid: int, logo_id: int,
                          width: int, height: int, bits: str) -> bytes:
    logo_id = max(1, min(5, int(logo_id)))
    data = (
        bytes([logo_id])
        + f"{width:04d}".encode("ascii")
        + f"{height:02d}".encode("ascii")
        + bits.encode("ascii")
    )
    return _frame(pkgid, ord("L"), data)
```

7. Exact Send Sequence

This is the central process that sends the text first, then the QR logo bitmap.

```
def push_label(self, text_payload: str, qr_payload: str) -> bool:
    self._pending_label_text = text_payload or ""
    self._pending_qr_payload = qr_payload or ""

    if self._status.connected and not self._status.simulated:
        try:
            # 1) Send QR payload as Network Text field 1.
            wire_text = _strip_for_text(qr_payload or text_payload or " ")
            self._send_recv(
                self._stamped(build_update_network_text, 1, wire_text),
                suppress_log=True,
            )

            # 2) Wait before sending logo bitmap.
            time.sleep(0.1)

            # 3) Generate QR bitmap.
            from core.qr_builder import build_printer_bitmap
            msg_print_dots = int(getattr(config_app, "MSG_PRINTED_DOTS", MAX_DOTS))
            qr_max = max(1, min(
                MAX_DOTS,
                int(getattr(config_app, "QR_MAX_DOTS", MAX_DOTS)),
                msg_print_dots if msg_print_dots > 0 else MAX_DOTS,
            ))
            qr_border = max(0, int(getattr(config_app, "QR_BORDER", 2)))
            qr_ecc = getattr(config_app, "QR_ERROR_CORRECTION", "L")

            w, h, bits = build_printer_bitmap(
                qr_payload or " ",
                max_size=qr_max,
                border=qr_border,
                error_correction=qr_ecc,
            )

            # 4) Send QR bitmap as Net Logo slot 1.
            self._send_recv(
                self._stamped(build_net_logo_packet, 1, w, h, bits),
                suppress_log=True,
            )
        except Exception as e:
            self.log.emit(f"Push failed: {e}")
            return False

    self.pushed.emit(qr_payload or "")
    return True
```

8. TCP Send Method

Current operation uses TCP. The final packet bytes are sent with socket.sendall(...).

```
def _send_recv_tcp(self, pkt: bytes, timeout_s: float) -> bytes:
    if self._sock is None:
        self._reopen_tcp_with_resync(timeout_s)
    self._sock.settimeout(timeout_s)
    self._sock.sendall(pkt)
    return self._read_tcp_frame(self._sock, timeout_s)
```

9. Message Parameters Currently Used

These settings affect how the active message is printed. We need confirmation whether these are correct for a scannable QR when the QR is uploaded as Net Logo.

```
QR_MAX_DOTS         = 30
QR_BORDER           = 2
QR_ERROR_CORRECTION = "L"

MSG_WIDTH           = 999
MSG_DELAY           = 100
MSG_HEIGHT          = 1
MSG_PRINTED_DOTS    = 30
MSG_TRIG_TIMES      = 1
MSG_GAP             = 0
MSG_COL_REPEATS     = 1
MSG_CHAR_SPACE      = 1
```

10. Questions For DIKAI Manufacturer

Please answer these questions so we can print a reliable, scannable QR on the DC810.

- Can the printer's 2D Code object receive dynamic QR data directly from our device over TCP? If yes, please provide the exact protocol command, packet format, field number, and example bytes.
- If direct 2D Code access is not supported, can the 2D Code object be configured to automatically use the dynamic value from Net Text field 1 as its QR content?
- Inside the printer's 2D Code editor, is there a way to bind the 2D Code content to Net Text or Network Text? We could not find this option in the menu.
- If the only supported external method is Net Logo upload, what exact Net Logo dimensions, printed dots, message width, message height, column repeats, and other message parameters should we use for a scannable QR?
- For QR strings longer than 30 characters, what is the recommended minimum printed dot size or QR version setting for this printhead?
- Does the DC810 firmware resize or stretch Net Logo images based on message parameters? If yes, which settings must be fixed to keep the QR square?
- Is error correction level L recommended for this printer, or should we use M/Q/H for better scan reliability?
- Do you recommend using a printer-native 2D Code field instead of Net Logo for QR? If yes, please provide the complete setup and communication method.

11. Message To Manufacturer

We are operating a DIKAI DC810 printer using TCP communication. Our application sends dynamic text and QR data to the printer. Net Text is working correctly. For the QR, our device currently generates the QR as a software bitmap and sends it to the printer as Net Logo slot 1. The printer receives it, but the printed QR quality is poor and QR scanners cannot read it reliably, especially when the payload string is longer than 30 characters.

We want to know whether the DC810 2D Code object can receive dynamic QR data directly from our device over TCP. If this is possible, please provide the exact command, packet structure, and example.

If direct TCP update of the 2D Code object is not possible, can the 2D Code field automatically use the value from Net Text field 1 as its QR content? We could not find a setting in the 2D Code editor to bind it to Net Text.

If Net Logo is the only available method, please advise the exact QR bitmap size, message parameters, print dots, width, height, column repeat, quiet zone, and error correction settings required for a clear and scannable QR on the DC810.

End of technical summary.